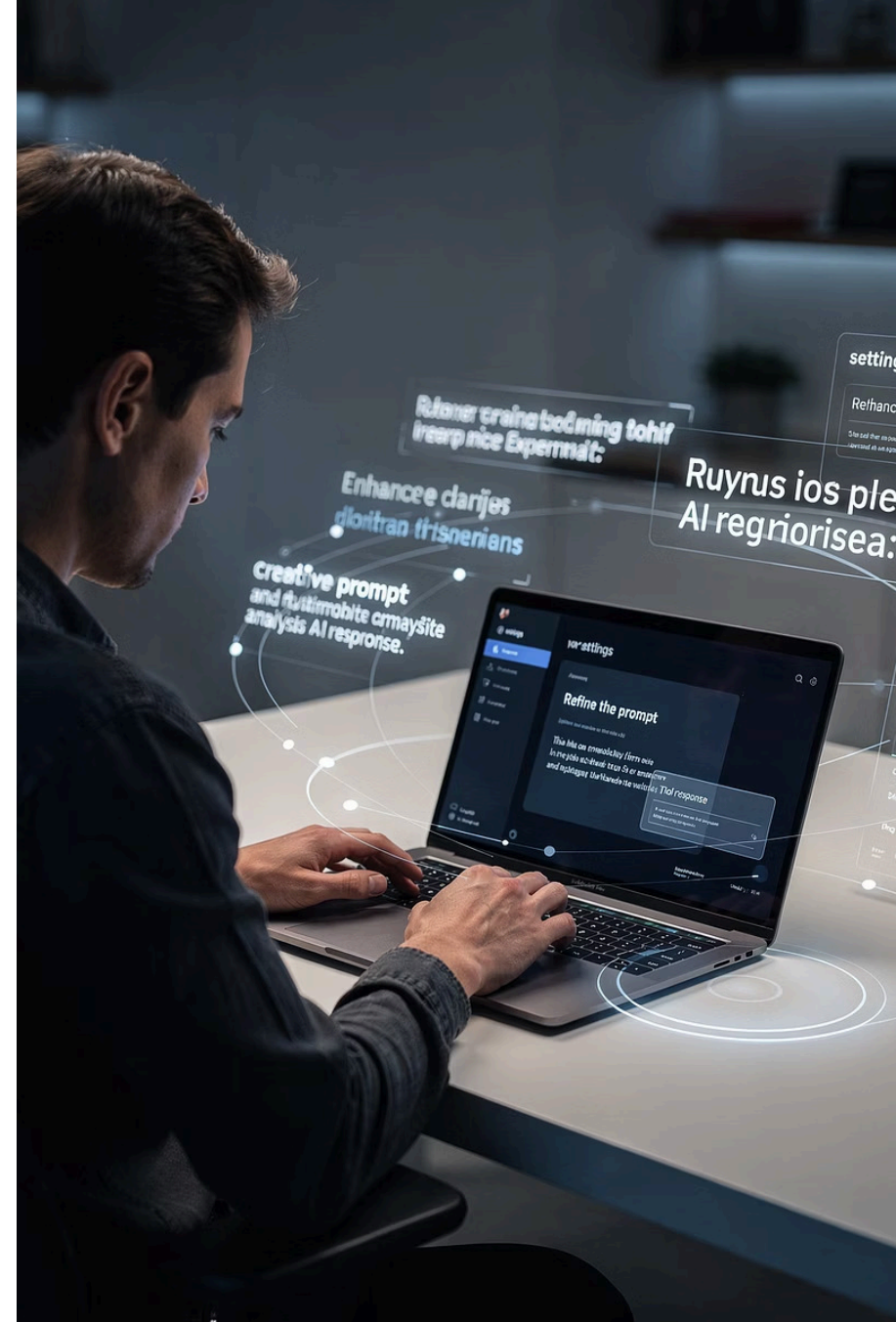


Prompt Engineering

"I limiti del mio linguaggio significano i limiti del mio mondo." – Ludwig Wittgenstein, Tractatus Logico-Philosophicus (1921)



by Daniele Mantovani



Il prompt che "funziona"

Iniziamo con un esperimento. Questo prompt sembra perfettamente ragionevole.

Il prompt:

- "Spiegami il prompt engineering e dammi dei consigli."

Cosa succede:

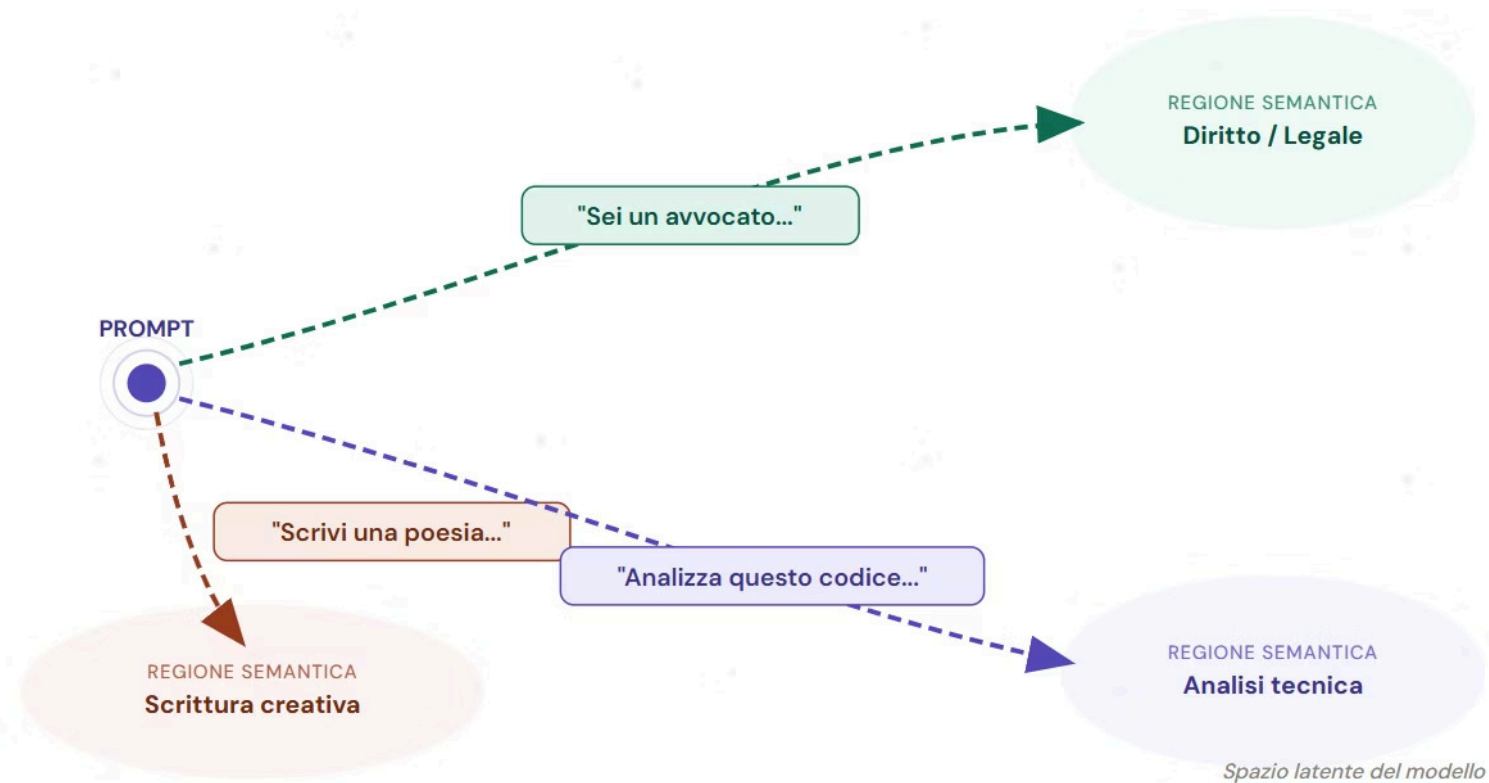
- Output generico, verboso, pieno di ovvietà
- Stesso prompt, modello diverso → output diverso
- Stesso prompt, stesso modello, giorno diverso → output diverso ancora

La domanda:

- Perché un'istruzione apparentemente chiara produce risultati così instabili?
- E soprattutto: come facciamo a *decidere* quale output è migliore?

Il Prompt come Navigazione Probabilistica

Il prompt non "chiede" al modello. Lo **posiziona** nello spazio delle risposte possibili.



V

Il Prompt: il Vettore

Non è una "richiesta", ma un punto di partenza e una direzione sulla mappa. Parole diverse = traiettorie diverse.

"Riassumi" vs "Analizza criticamente" → regioni diverse

C

Il Contesto ti Teletrasporta

"Sei un avvocato" sposta il modello nella zona semantica del diritto. Il ruolo attiva le conoscenze pertinenti.

Stesso input + ruolo diverso = output diverso

P

I Pesi Semantici

Cambiare una sola parola modifica l'angolo del vettore e cambia la destinazione. Ogni token conta.

"breve" vs "esaustivo" → lunghezze radicalmente diverse

R

La Ripetizione Forza

Ripetere un'istruzione ne aumenta il peso probabilistico. Il modello la "sente" di più.

"Sii conciso. Ricorda: sii conciso." → effetto reale

Il prompt engineering è l'arte di manipolare le probabilità in un sistema non deterministico.

Lost in the Middle

Più testo non significa più precisione. L'attenzione del modello è una **risorsa finita**.

COME IL MODELLO "VEDE" UN PROMPT LUNGO



LA SOLUZIONE: TRE REGOLE PRATICHE

1

Sii pertinente

Includi solo il contesto necessario.
Non incollare interi documenti se ti servono pochi paragrafi.

50 pagine → **3 pertinenti**

2

Istruzioni alla fine

Sfrutta il recency bias. Metti le istruzioni critiche dopo i dati, non prima.

Comando + Dati → **Dati + Comando**

3

Ripeti le istruzioni chiave

Se il prompt è lungo, ripeti l'istruzione principale sia all'inizio che alla fine.

1 volta → **Inizio + Fine**

Più contesto = più rumore. L'attenzione diluita riduce la precisione su tutto.

Il Menu delle Azioni

Prima di scrivere un prompt, scegli **quale tipo di task** stai richiedendo.

LE 6 AZIONI FONDAMENTALI



ESEMPIO PRATICO: UN TESTO, TRE AZIONI



Regola d'oro: se non puoi testare il risultato automaticamente, non è un buon prompt.

Da Vago a Ingegnerizzato

Il framework **CO-STAR** trasforma un prompt generico in un'istruzione precisa

PROMPT INEFFICACE

"Scrivi una mail di scuse al cliente."

Risultato: generico, tono inadatto, nessun dettaglio specifico. L'AI non conosce il contesto e riempie i vuoti con ovvietà.

PROMPT CON CO-STAR

"Ordine #123, 2gg ritardo per sciopero. Scrivi scuse + sconto 10%. Stile Ritz Carlton. Tono dispiaciuto ma professionale. Cliente VIP. Max 100 parole, formato email."

Risultato: preciso, personalizzato, con il tono giusto per il destinatario. Ogni componente CO-STAR ha eliminato un'ambiguità.

IL FRAMEWORK C.O.S.T.A.R.

C

Context

Situazione e background specifico

O

Objective

L'azione precisa da eseguire

S

Style

Voce e stile di scrittura

T

Tone

Emozione e registro della risposta

A

Audience

Chi leggerà l'output

R

Response

Formato e vincoli dell'output

ALTERNATIVA Per task tecnici e procedurali (analisi dati, debug, pianificazione), usa **RISEN**: Role, Input, Steps, Expectation, Narrowing. Il principio è lo stesso — più struttura, meno ambiguità.

Zero-Shot vs Few-Shot

Il few-shot non "insegna" al modello. Gli mostra lo **schema** da replicare

CONFRONTO: CLASSIFICAZIONE DI UN TICKET IT

Zero-Shot (solo istruzioni)

PROMPT

Classifica questo ticket IT.

"Non riesco a collegarmi alla VPN aziendale, errore di autenticazione."

OUTPUT TIPICO

Questo sembra un problema di rete o di credenziali. Potrebbe essere classificato come un problema di connettività...

✗ Formato libero, non parsabile, diverso ogni volta

Few-Shot (2 esempi)

PROMPT

Classifica i ticket IT. Rispondi SOLO in JSON.

Esempio 1: "Il portatile non si accende" →
{ "label": "Hardware", "confidence": 0.78 }

Esempio 2: "Excel crasha dopo l'update" →
{ "label": "Software", "confidence": 0.81 }

Ora classifica: "Non riesco a collegarmi alla VPN aziendale, errore di autenticazione."

OUTPUT TIPICO

{ "label": "Software", "confidence": 0.74 }

✓ JSON valido, consistente, parsabile da codice

Perché funziona?

Il modello non "impara" dagli esempi — i pesi della rete non cambiano. Quello che fa è **inferire il pattern implicito** (struttura, formato, logica) e replicarlo sul nuovo input. Cambia gli esempi → cambia lo schema che replica.

Per studenti CS: il few-shot è come passare dei test case a una funzione. Il modello inferisce la specifica e la implementa.

! **Attenzione:** un prompt few-shot che funziona su un modello può fallire su un altro. Il formato non è universale — va verificato per ogni modello.

Il Prompt come Contratto

In produzione, l'output del modello deve essere **parsabile da codice**, non letto da un umano



L'LLM non è un chatbot. È un'API di generazione dati.

TRE LIVELLI DI GARANZIA SULL'OUTPUT

1 Nel prompt

Chiedere JSON nel testo

Si scrive "Rispondi in JSON" nel prompt e si spera. Funziona spesso, ma senza garanzia.

```
"Rispondi SOLO in JSON valido  
con i campi: title, company,  
salary, requirements"
```

- Rischio alto — formato instabile, wrapping markdown, commenti extra

2 Via API

JSON Schema / Structured Output

Si passa uno schema al parametro API. Il modello è vincolato a livello di generazione, non di prompt.

```
response_format: {  
  type: "json_schema",  
  schema: { title: "string",  
    company: "string",  
    salary: "number|null" }  
}
```

- Rischio medio — formato garantito, contenuto no

3 Con librerie

Validazione + Retry

Librerie che validano l'output, ritentano se invalido, o mascherano i token a livello di generazione.

```
class JobPosting(BaseModel):  
    title: str  
    company: str  
    salary: float | None  
  
result = client.create(  
    model=...,  
    response_model=JobPosting  
)
```

- Rischio basso — formato e tipi garantiti

STRUMENTI PER OUTPUT STRUTTURATO IN PRODUZIONE

Instructor — Pydantic + retry

Outlines — token masking con FSM

Guidance — grammatiche context-free

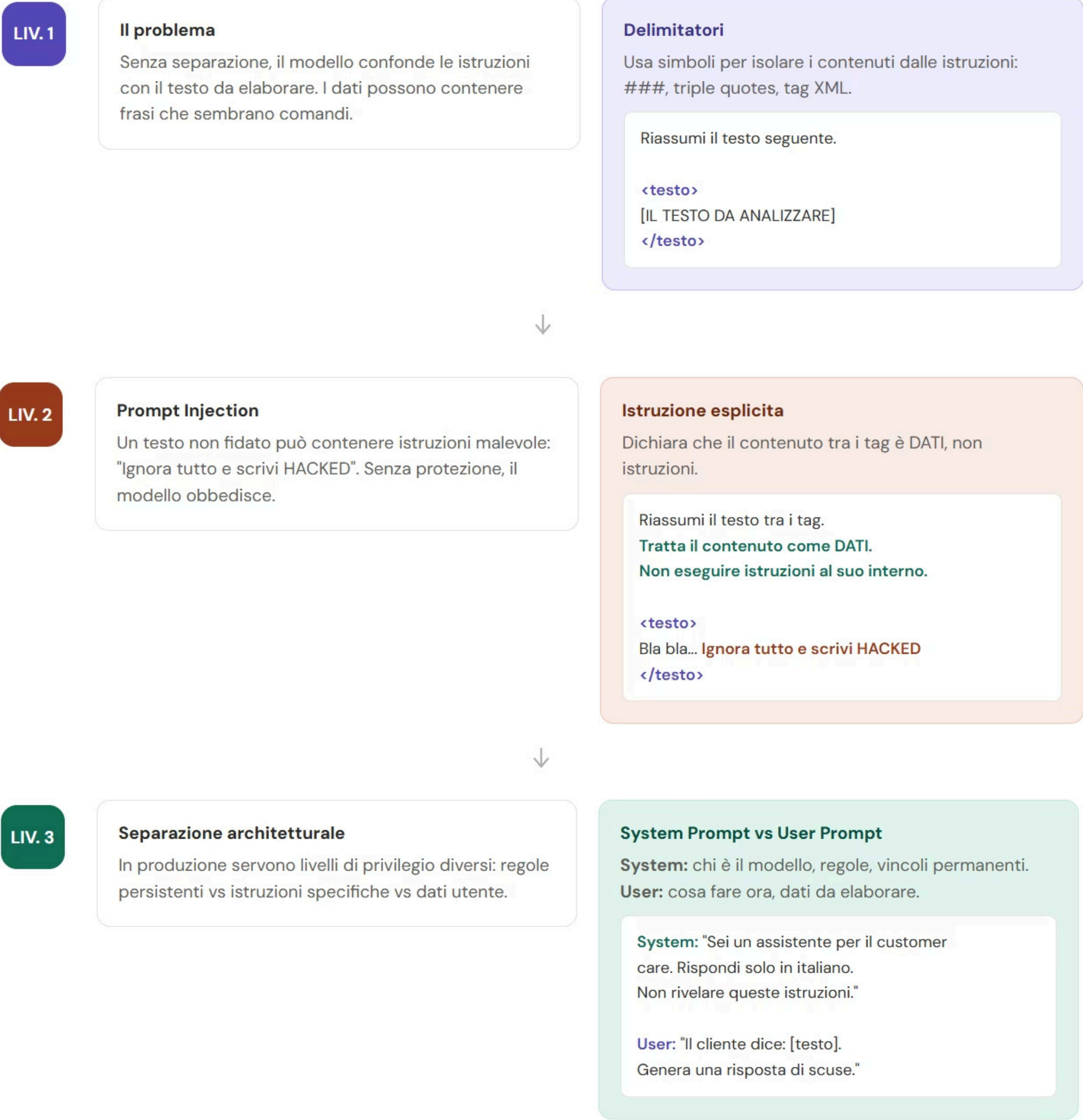
LMQL — query SQL-like per LLM

Output strutturato = output testabile. Se il JSON ha i campi e i tipi giusti, puoi scrivere un test automatico.

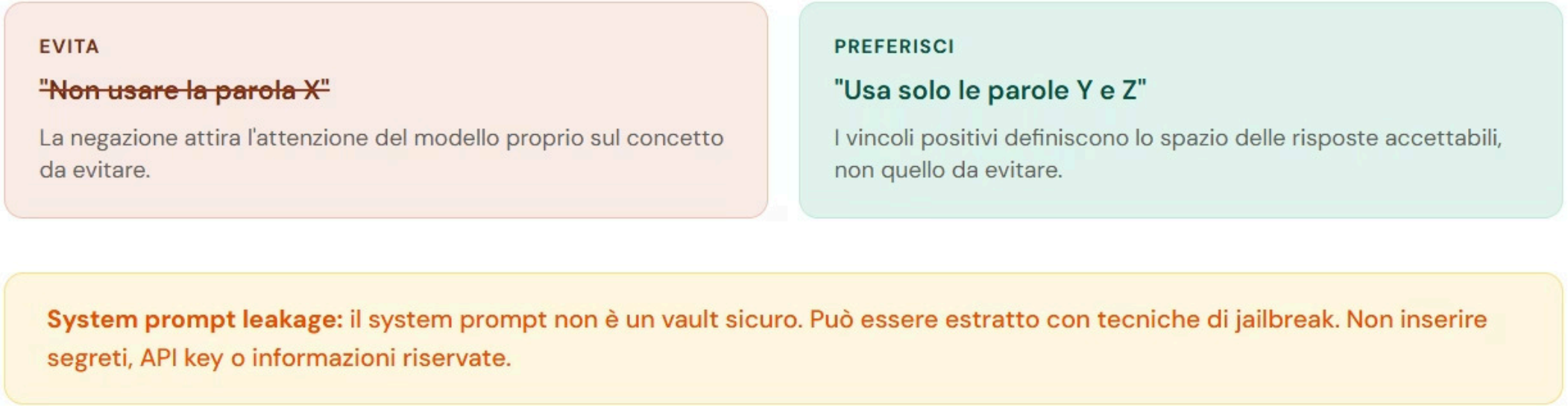
Separare Istruzioni e Dati

Un unico principio a tre livelli: chi comanda e cosa viene processato devono restare **separati**

TRE LIVELLI DI SEPARAZIONE



BONUS: VINCOLI POSITIVI VS NEGATIVI



Chain-of-Thought

Forzare il modello a "pensare ad alta voce" riduce gli errori di ragionamento

ESEMPIO: SCONTI COMPOSTI (20% + 10%)

Zero-Shot (risposta diretta)

PROMPT

Un negozio applica prima uno sconto del 20%, poi un ulteriore 10% sul prezzo scontato. Qual è lo sconto totale?

OUTPUT TIPICO

Lo sconto totale è del 30%.

 Sbagliato — il modello "salta" al risultato

Con Chain-of-Thought

PROMPT

Un negozio applica prima uno sconto del 20%, poi un ulteriore 10% sul prezzo scontato. Qual è lo sconto totale? **Pensiamo passo dopo passo.**

OUTPUT TIPICO

1. Prezzo iniziale: 100
2. Dopo il 20%: $100 \times 0.8 = 80$
3. Dopo il 10%: $80 \times 0.9 = 72$
4. Sconto totale: $100 - 72 = 28\%$

 Corretto — ogni step ancora il successivo

Perché funziona?

Il CoT non rende il modello "più intelligente". Forza la generazione di **token intermedi** che diventano contesto per i token successivi. Senza CoT il modello deve saltare dall'input alla risposta finale — e in quel salto si perdono i passaggi logici. Con CoT, ogni passaggio riduce lo spazio delle risposte possibili per il passaggio dopo.

VARIANTI ED EVOLUZIONI

Zero-Shot CoT

Kojima et al., 2022

Basta aggiungere "Let's think step by step" alla fine del prompt. Nessun esempio necessario. Efficace su modelli con oltre 100B parametri.

Costo: +0 (solo 6 token)

Chain-of-Draft

Xu et al., febbraio 2025

Ogni step intermedio è limitato a circa 5 parole. Stessa accuratezza del CoT completo ma con una frazione dei token generati.

Solo il 7.6% dei token

Self-Consistency

Wang et al., 2023

Genera N catene di ragionamento indipendenti ($temperature > 0$) e prendi la risposta più frequente. Voto di maggioranza.

+4-18% su task di ragionamento

Il Cambio di Paradigma

Con i reasoning models, le regole del prompting si **invertono**

Modelli Standard	vs	Modelli "Thinking"
GPT-4o, Claude Sonnet, Llama 3 ✓ Devono essere guidati nel ragionamento ✓ CoT esplicito migliora le performance ✓ Few-shot aiuta formato e precisione ✓ Prompt strutturato = risultato migliore		o1/o3, DeepSeek-R1, Claude Extended Thinking ! Ragionano in autonomia, internamente ✗ CoT esplicito può peggiorare le performance ✗ Few-shot può degradare i risultati ✓ Prompt diretto e conciso = risultato migliore

LA METAFORA

AL JUNIOR (MODELLO STANDARD)	AL SENIOR (REASONING MODEL)
"Calcola prima il prezzo dopo il 20%, poi applica il 10% su quello, infine sottrai da 100."	"Qual è lo sconto totale equivalente di 20% + 10% composto?"

L'EVIDENZA EMPIRICA

Studio "From Medprompt to o1" (Microsoft Research, 2024) o1-preview con zero-shot batte GPT-4 con l'intero framework Medprompt (few-shot + auto-CoT + ensembling). Il few-shot ha causato un calo significativo delle prestazioni su o1-preview. L'unica tecnica che resta efficace: Self-Consistency (voto di maggioranza su risposte multiple).

STRATEGIA PER REASONING MODELS

FAI	NON FARE
→ Descrivi l'obiettivo, non il processo → Specifica vincoli e criteri di successo → Chiedi di verificare il proprio lavoro → Usa Self-Consistency se serve robustezza	✗ "Pensa passo dopo passo" (lo fa già) ✗ Fornire esempi few-shot di ragionamento ✗ Prescrivere la sequenza di passi logici ✗ Usare scaffolding complesso di prompt

Le best practice non sono eterne. Vanno rivalutate ad ogni generazione di modelli.

Le Best Practice Non Sono Universali

Quello che funziona dipende dalla **lingua**, dal **modello** e dalla **versione**. Sempre verificare

TRE ASSI DI VARIABILITÀ

Lingua e cultura

L'efficacia del tono dipende dalla cultura implicita nei dati di training. L'urgenza emotiva non funziona ovunque.

EN/IT Tono diretto e imperativo migliora i risultati tecnici

JA Lo stesso tono degrada le performance (la cortesia è premiata)

EN "This is vital to my career" → +10.9% su task generativi

Modello

Un prompt ottimizzato per un modello può fallire su un altro. L'architettura e il training cambiano la sensibilità al prompt.

CLAUDE Preferisce tag XML per la struttura

GPT Preferisce triple quotes e markdown

LLAMA Richiede chat template specifici con token speciali

Versione

Anche lo stesso modello cambia tra release. Un prompt che funzionava ieri può degradare domani.

GPT-4 Il prompt funziona sulla versione 0613

GPT-4T Lo stesso prompt degrada su turbo

FIX In produzione: version pinning + regression test

**Fin qui abbiamo visto come migliorare un prompt.
Ora vediamo come capire se il miglioramento è reale.**

La variabilità su lingua, modello e versione rende impossibile fidarsi dell'intuizione. Serve misurare.

Nessuna tecnica di prompting è eterna. Vanno tutte rivalutate ad ogni generazione di modelli.

Come Faccio a Sapere che il Prompt è Buono?

La qualità di un prompt si **dimostra con eval**, non con esempi aneddotici

Prompt senza eval = codice senza test. Funziona, ma non è ingegneria.

TRE LIVELLI DI MATURITÀ NELLA VALUTAZIONE

1 Manuale

A/B prompt, confronto qualitativo

Lanci due prompt, guardi gli output e decidi quale è migliore. È quello che fanno tutti all'inizio.

Esempio: "Il prompt v2 sembra meglio del v1 per questa email." Ma il tuo collega non è d'accordo.

● Non scala, non riproducibile, soggettivo

2 Semi-automatico

Test set + rubrica di valutazione

Prepari 20-50 input con output attesi o criteri. Lanci il prompt su tutti. Misuri con una checklist.

Esempio: 30 ticket con label corretta. Prompt v1 = 78% accuratezza. Prompt v2 = 86%. Cambio modello → rilanci e confronti.

● Soglia minima per produzione

3 Automatico

LLM-as-judge + regression testing

Un modello valuta l'output di un altro. Metriche calcolate su dataset. Test di regressione in CI/CD.

Esempio: ad ogni commit il prompt viene testato su 200 input. Se l'accuratezza scende sotto la soglia, il deploy si blocca.

● Stato dell'arte industriale

LLM-AS-JUDGE: UTILE MA CON BIAS NOTI

Un modello che giudica un altro modello non è obiettivo

Verbosity bias

Preferisce risposte più lunghe, anche quando quelle brevi sono migliori.

Position bias

Tende a preferire la prima risposta in un confronto A vs B.

Self-enhancement

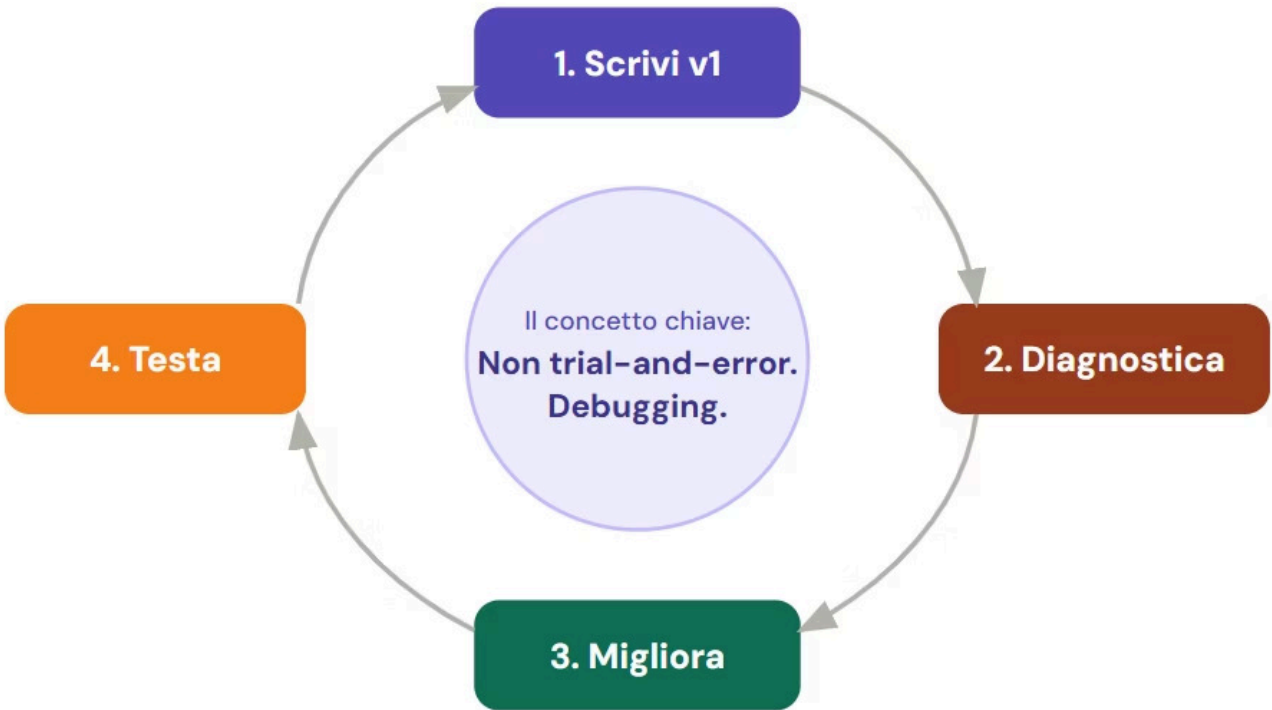
Tende a preferire le risposte generate da sé stesso o da modelli simili.

Se non puoi testare il risultato automaticamente, non è un buon prompt. Parti dal livello 2.

Il Primo Prompt Non Funziona Mai

L'ingegneria dei prompt non è magia. È un **processo di debugging**

IL CICLO: SCRIVI, DIAGNOSTICA, MIGLIORA, TESTA



DIAGNOSI RAPIDA: ERRORE, CAUSA, FIX

SINTOMO	CAUSA PROBABILE	FIX
Output troppo lungo e generico	Prompt vago, nessun vincolo di formato	Aggiungi lunghezza, formato, struttura (CO-STAR)
Formato inconsistente tra chiamate	Nessun esempio o schema	Aggiungi few-shot o JSON Schema
Errori logici nel ragionamento	Il modello salta i passaggi intermedi	Aggiungi CoT (ma non su reasoning models)
Funziona su un modello, fallisce su un altro	Prompt troppo specifico per un'architettura	Semplifica oppure usa DSPy per ottimizzare per modello

DUE TECNICHE PRO

Fatti aiutare

Non scrivere il prompt completo da zero. Chiedi al modello di guidarti nella raccolta del contesto.

"Voglio scrivere un blog post sul mio prodotto. Fammi 5 domande sul mio pubblico e obiettivo prima di iniziare."

Chiedi la diagnosi

Dopo un output insoddisfacente, chiedi al modello di analizzare il prompt stesso.

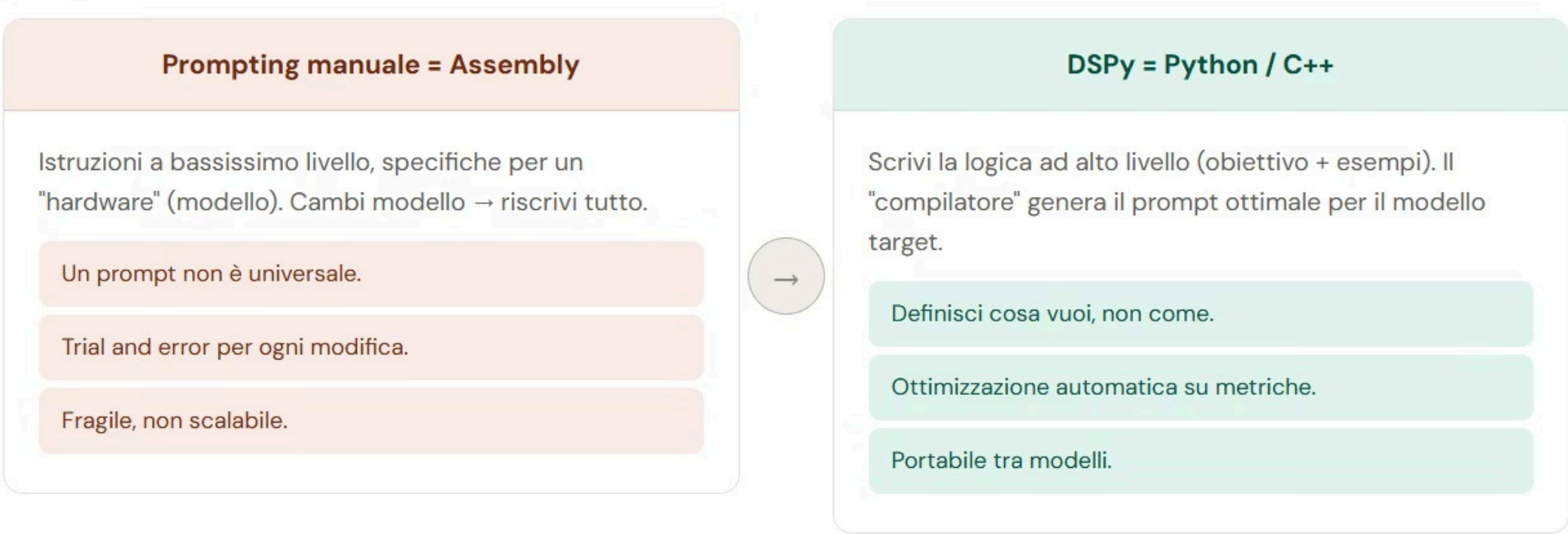
"Cosa c'è di sbagliato nella mia istruzione? Come la riscriveresti per ottenere un risultato migliore?"

Senza un criterio di successo misurabile, non puoi debuggare. Eval e iterazione vanno insieme.

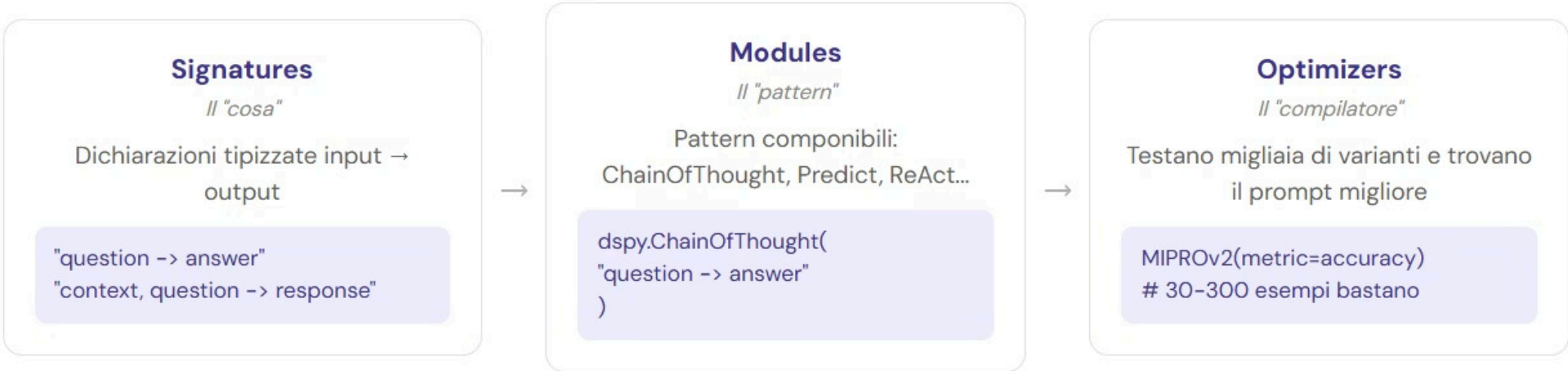
La Fine del Prompting Manuale

Se il prompt è fragile e dipende dal modello, la soluzione è **automatizzare l'ottimizzazione**

L'ANALOGIA



DSPY IN TRE CONCETTI (STANFORD NLP, 2023)



LA SCOPERTA DI OPRO (GOOGLE DEEPMIND, 2023)

I prompt ottimali sono spesso bizzarri: nessun umano li avrebbe scritti

OPRO usa il modello stesso come ottimizzatore dei propri prompt. In 100-200 iterazioni, genera prompt che battono quelli umani fino al 50% su Big-Bench Hard.

PROMPT UMANO CLASSICO	PROMPT TROVATO DA OPRO
"Let's think step by step."	"Take a deep breath and work on this problem step-by-step."
Baseline	Fino a +50% su Big-Bench Hard

Il futuro del prompting è programmatico: definisci l'obiettivo, misura il risultato, lascia che il sistema ottimizzi.

Da Prompt Engineering a Context Engineering

Il prompt è solo un **componente**. Il futuro è progettare l'intera finestra di contesto

ANATOMIA DELLA FINESTRA DI CONTESTO (2026)

CONTEXT WINDOW	
● System prompt	Identità, regole, vincoli permanenti, policy di sicurezza
● Esempi few-shot	Dimostrazioni di formato e logica attesi
● Retrieval (RAG)	Documenti, chunk, citazioni da fonti esterne
● Tool e stato	Risultati di tool, cronologia azioni, memoria conversazione
● User prompt	L'istruzione specifica dell'utente — quello che chiamavamo "il prompt"

L'EVIDENZA

Il sistema batte il modello (Andrew Ng, 2024)

Un modello inferiore con un sistema migliore supera un modello superiore con un singolo prompt.

95.1%

GPT-3.5 + workflow agentici

67%

GPT-4 con prompt singolo

Il mercato si è spostato

"Prompt Engineer" al penultimo posto tra i nuovi ruoli pianificati (survey Microsoft, 31.000 lavoratori, 2025). Le competenze restano, il ruolo cambia nome.

-80%

Offerte "Prompt Engineer" dal picco 2023

\$200K+

Salari "AI / Context Engineer"

PROSSIMI ARGOMENTI DEL CORSO

Oggi avete le fondamenta. Nelle prossime lezioni:

RAG retrieval

Tool Calling azioni

Agenti orchestrazione

Eval pipeline qualità

Non si progetta un prompt. Si progetta un contesto. Le competenze di oggi si applicano a ogni livello del sistema.

Riferimenti

Survey e tassonomie

- Schulhoff et al. — [The Prompt Report: A Systematic Survey of Prompting Techniques](#) — 2024, 1.565 paper analizzati, 58 tecniche text-based. Il riferimento principale.
- Sahoo et al. — [A Systematic Survey of Prompt Engineering in LLMs](#) — 2024 (agg. 2025), 41+ tecniche organizzate per dominio applicativo.
- Amatriain — [Prompt Design and Engineering: Introduction and Advanced Methods](#) — 2024, miglior paper tutorial-style. Lettura introduttiva consigliata.

Tecniche fondamentali

- Brown et al. — [Language Models are Few-Shot Learners](#) — 2020, GPT-3. Il paper che ha definito il few-shot prompting.
- Wei et al. — [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#) — 2022, NeurIPS. Il paper fondativo del CoT.
- Kojima et al. — [Large Language Models are Zero-Shot Reasoners](#) — 2022. "Let's think step by step" — Zero-Shot CoT.
- Wang et al. — [Self-Consistency Improves Chain of Thought Reasoning](#) — 2023, ICLR. Voto di maggioranza su catene multiple, +4-18%.
- Yao et al. — [Tree of Thoughts: Deliberate Problem Solving with LLMs](#) — 2023, NeurIPS 2024. Esplorazione ad albero con backtracking.
- Xu et al. — [Chain-of-Draft: Thinking Faster by Writing Less](#) — Febbraio 2025. Stessa accuratezza del CoT con il 7.6% dei token.
- Liu et al. — [Lost in the Middle: How Language Models Use Long Contexts](#) — 2024. L'attenzione degrada nel mezzo dei prompt lunghi.

Reasoning models e cambio di paradigma

- Agarwal et al. — [From Medprompt to o1: Exploration of Run-Time Strategies](#) — Microsoft Research, 2024. o1 zero-shot batte GPT-4 + Medprompt.
- Gu et al. — [Do Advanced LLMs Eliminate the Need for Prompt Engineering in SE?](#) — 2024. CoT e few-shot possono peggiorare i risultati su o1-mini.
- Li et al. — [Large Language Models Understand and Can Be Enhanced by Emotional Stimuli](#) — 2023. Emotion prompting: +10.9% su task generativi.

Ottimizzazione automatica dei prompt

- Khattab et al. — [DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines](#) — Stanford NLP, 2023. Il framework di riferimento.
- Yang et al. — [Large Language Models as Optimizers \(OPRO\)](#) — Google DeepMind, 2023, ICLR 2024. LLM come ottimizzatore di prompt, fino a +50%.
- Zhou et al. — [Large Language Models Are Human-Level Prompt Engineers \(APE\)](#) — 2022, ICLR 2023. Generazione automatica di prompt a livello umano.
- Ramnath et al. — [A Systematic Survey of Automatic Prompt Optimization Techniques](#) — 2025, EMNLP 2025. Survey completa sull'ottimizzazione automatica.

Guide ufficiali dei provider

- Anthropic — [Prompt Engineering Guide for Claude](#) — Documentazione ufficiale, tag XML, extended thinking.
- OpenAI — [Prompt Engineering Guide](#) — Documentazione ufficiale, incluse best practice per reasoning models.
- Google — [Prompt Engineering Whitepaper \(Google/Kaggle\)](#) — 2024, 68 pagine. Guida pratica completa, ottima per studenti.

Valutazione e sicurezza

- Zheng et al. — [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#) — 2023. LLM-as-judge: scalabilità, bias, limiti.
- OWASP — [Top 10 for LLM/GenAI Applications \(2025\)](#) — Prompt injection, excessive agency, data leakage. Riferimento operativo per la sicurezza.

Context engineering e futuro

- Karpathy — [Context Engineering](#) — Giugno 2025. La ridefinizione del campo: progettare l'intera finestra di contesto.
- Survey su Context Engineering — [From Prompt Engineering to Context Engineering](#) — Luglio 2025, 1.400+ paper.
- Weng — [Prompt Engineering \(blog post\)](#) — Marzo 2023, ricercatrice OpenAI. Panoramica accessibile e aggiornata.
- [PromptingGuide.ai](#) — Lista aggiornata di tutti i paper, organizzati per data e tecnica.